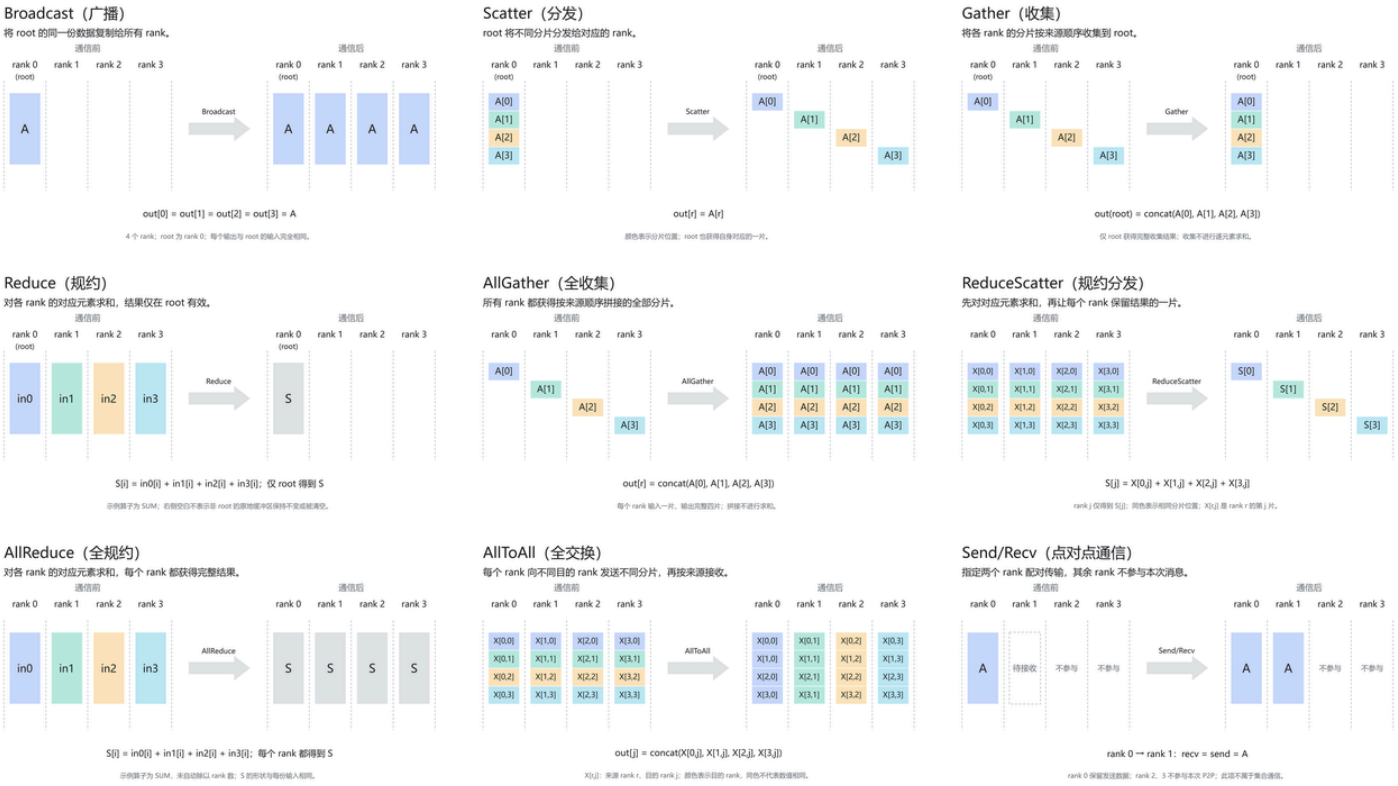
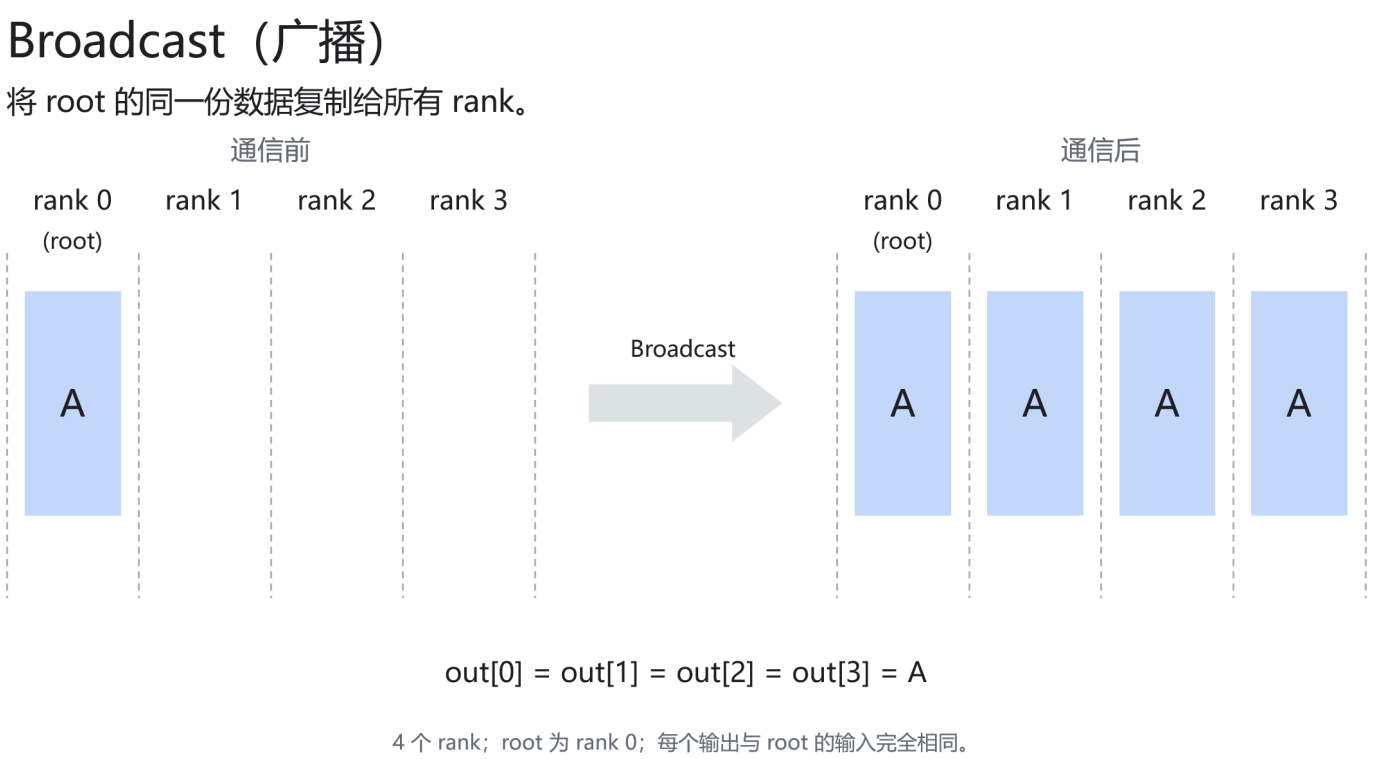
集合通信原语，并行训练通信行为



基础集体通信原语

通信原语	核心作用	大模型训练中的典型用途
Broadcast	一个 Rank 向所有 Rank 发送相同数据	DDP 参数初始化同步、模型状态广播
Reduce	多个 Rank 的数据规约到一个 Rank	全局 Loss、正确预测数、Token 数统计
Scatter	Root 将不同分片分发给不同 Rank	集中式数据分发、初始张量分片
Gather	多个 Rank 的分片集中到 Root	收集预测结果、评估输出、调试数据
AllGather	所有 Rank 收集完整数据	ZeRO-3/FSDP 参数重建、TP/SP 激活收集
ReduceScatter	跨 Rank 规约，每个 Rank 保留一部分	ZeRO-2/3 梯度分片、TP+SP 梯度通信
AllReduce	跨 Rank 规约，所有 Rank 获得相同结果	DDP 梯度同步、TP 局部计算结果求和
Send/Recv	两个 Rank 间定向发送数据	PP 流水线阶段间传输激活与梯度
AllToAll	所有 Rank 相互交换不同的数据分片	MoE Token 路由、Ulysses 序列并行

01 Broadcast：广播



核心定义：一对多的单向通信。从一个根节点（Root Rank），把完全相同的数据，发送给通信域内所有其他rank，最终所有rank都持有和root完全一致的数据。

用途：DDP 初始化时保持参数/缓冲区一致；分发配置信息或部分训练状态。

数据流：

代码块

```
初始状态：
Rank0: [A0, A1, A2, A3]  （root，持有完整数据）
Rank1: []
Rank2: []
Rank3: []

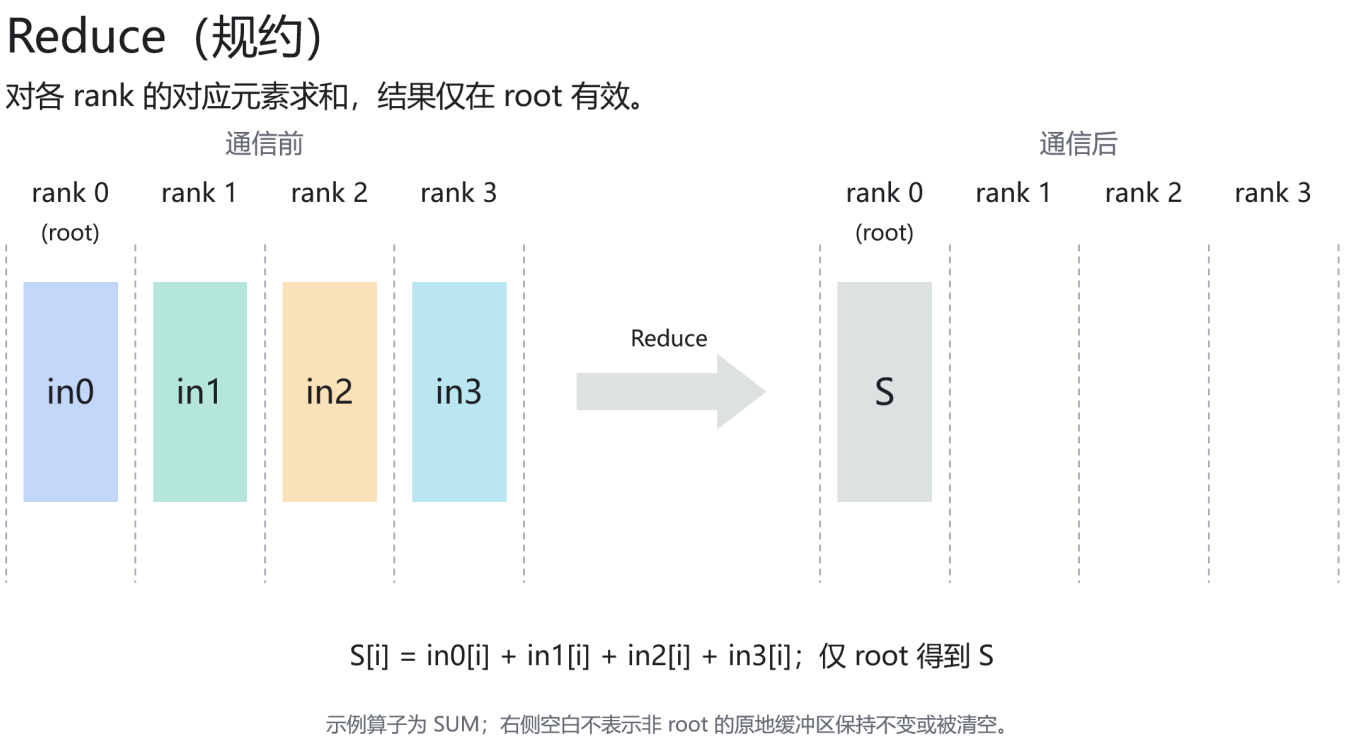
广播后状态：
Rank0: [A0, A1, A2, A3]
Rank1: [A0, A1, A2, A3]
Rank2: [A0, A1, A2, A3]
Rank3: [A0, A1, A2, A3]
```

代码块

```
x = T([0, 1, 2, 3]) if r == 0 else T([-1] * 4)
dist.broadcast(x, src=0)
```

```
print(r, x.tolist())
```

02 Reduce：规约



核心定义：多对一的通信。通信域内所有rank的输入数据，按指定规约操作（如SUM）聚合，最终把聚合结果**只发送给根节点Root Rank**。

用途：计算全局的loss，全局指标统计。把损失总和、正确数和样本数汇总到 root，再计算全局均值或准确率。有效 token 数不等时，应汇总总量和计数后再相除。

数据流：

代码块

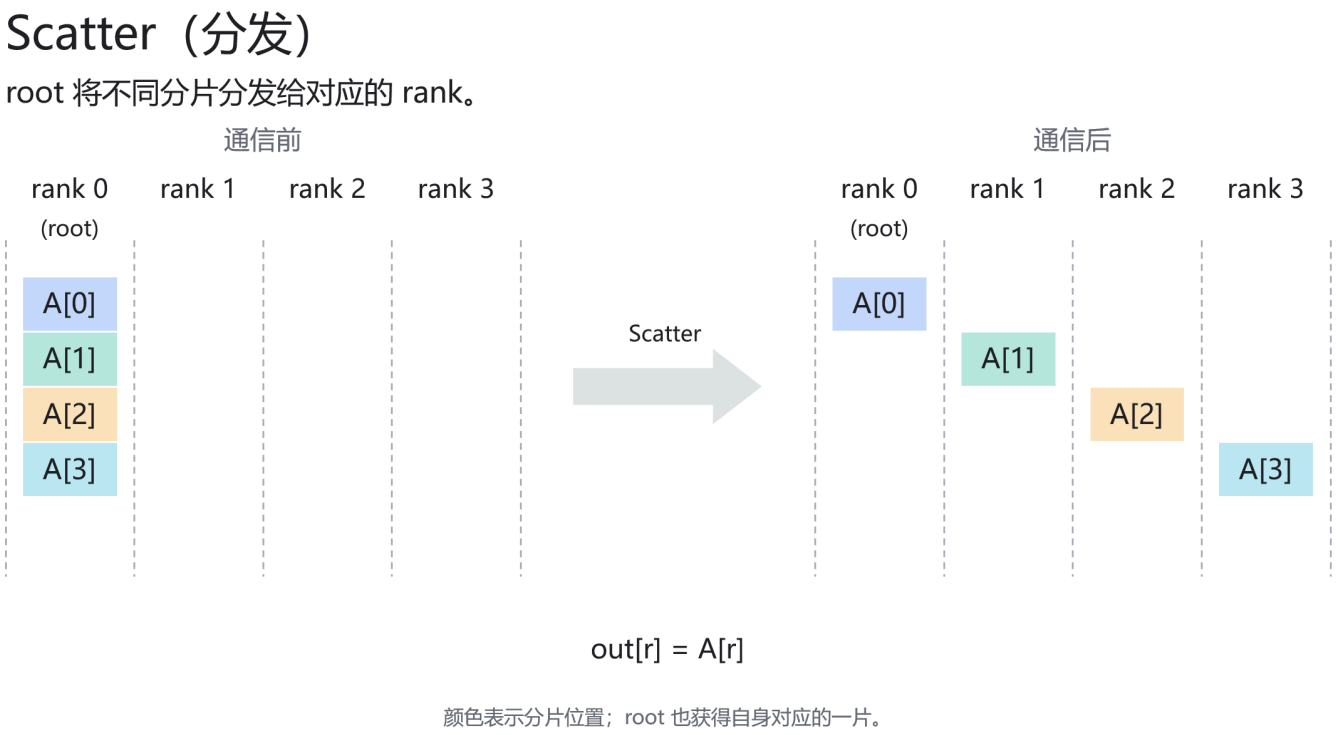
```
初始状态：
Rank0: [A0, A1, A2, A3]
Rank1: [B0, B1, B2, B3]
Rank2: [C0, C1, C2, C3]
Rank3: [D0, D1, D2, D3]

Reduce后状态：
Rank0: [A0+B0+C0+D0, A1+B1+C1+D1, A2+B2+C2+D2, A3+B3+C3+D3]（root，拿到聚合结果）
Rank1: [B0, B1, B2, B3]  （数据不变）
Rank2: [C0, C1, C2, C3]  （数据不变）
Rank3: [D0, D1, D2, D3]  （数据不变）
```

代码块

```
x = T([r, r + 1, r + 2, r + 3])
dist.reduce(x, dst=0, op=dist.ReduceOp.SUM)
if r == 0:
    print(r, x.tolist())
```

03 Scatter：分发



核心定义：一对多的分片分发。根节点Root Rank把一个完整大张量，按rank数量切成等长的分片，把每个分片 **定向分发给对应的rank**，最终每个rank只拿到属于自己的那一个分片。

用途：把集中存储的数据定向分配给不同 GPU。但标准 PyTorch DDP 一般通过 `DistributedSampler` 和各 Rank 的 DataLoader 独立读取数据，而不是每一步由 Rank0 Scatter 数据。

数据流：

代码块

```
初始状态：
Rank0: [A0,A1,A2,A3, B0,B1,B2,B3, C0,C1,C2,C3, D0,D1,D2,D3] （root，完整张量）
Rank1: []
Rank2: []
Rank3: []

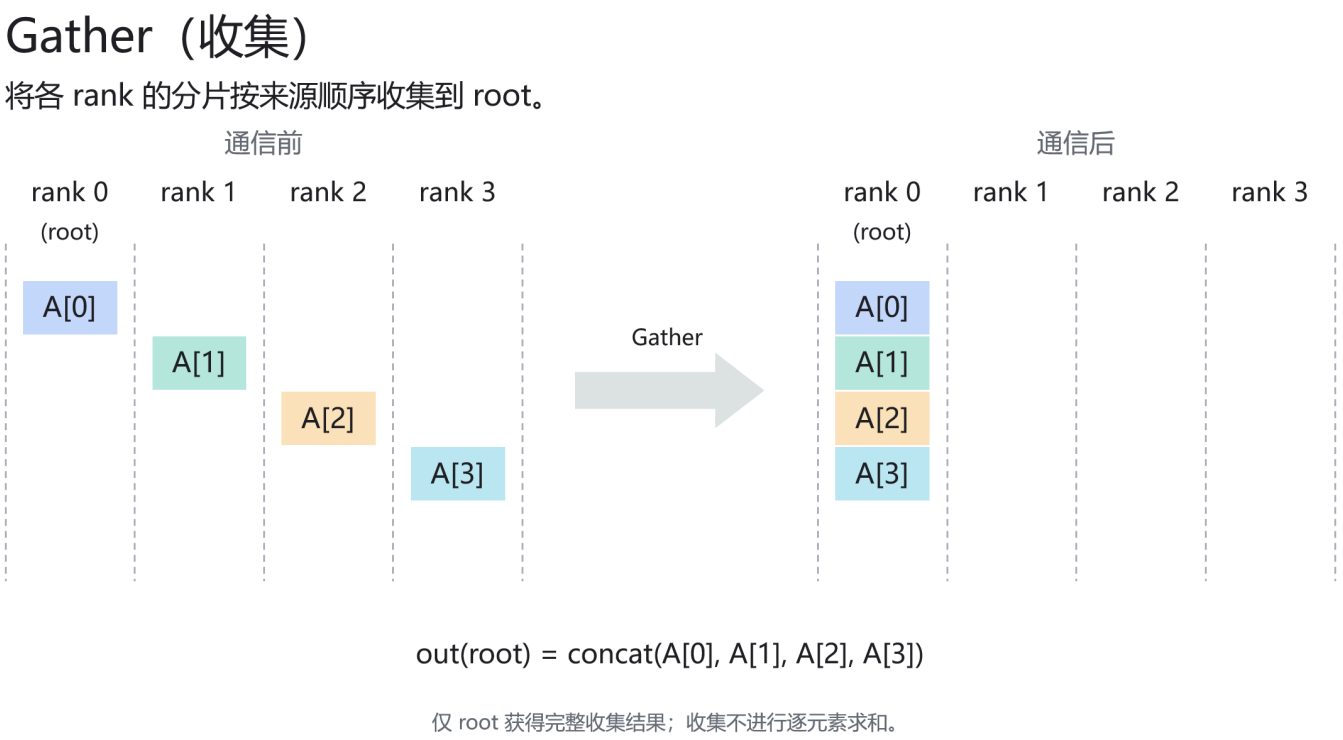
Scatter后状态：
Rank0: [A0, A1, A2, A3] （第0个分片）
Rank1: [B0, B1, B2, B3] （第1个分片）
Rank2: [C0, C1, C2, C3] （第2个分片）
```

```
Rank3: [D0, D1, D2, D3] （第3个分片）
```

代码块

```
out = T([0, 0])
parts = None
if r == 0:
    parts = [T([i, i + 10]) for i in range(p)]
dist.scatter(out, scatter_list=parts, src=0)
print(r, out.tolist())
```

04 Gather：收集



核心定义：Scatter的逆操作，多对一的分片收集。通信域内每个rank把自己持有的分片，发送给根节点Root Rank，root把所有分片按顺序拼接成完整的大张量。

用途：分布式推理结果汇总

数据流：

代码块

```
初始状态:
Rank0: [A0, A1, A2, A3]
Rank1: [B0, B1, B2, B3]
Rank2: [C0, C1, C2, C3]
Rank3: [D0, D1, D2, D3]

```

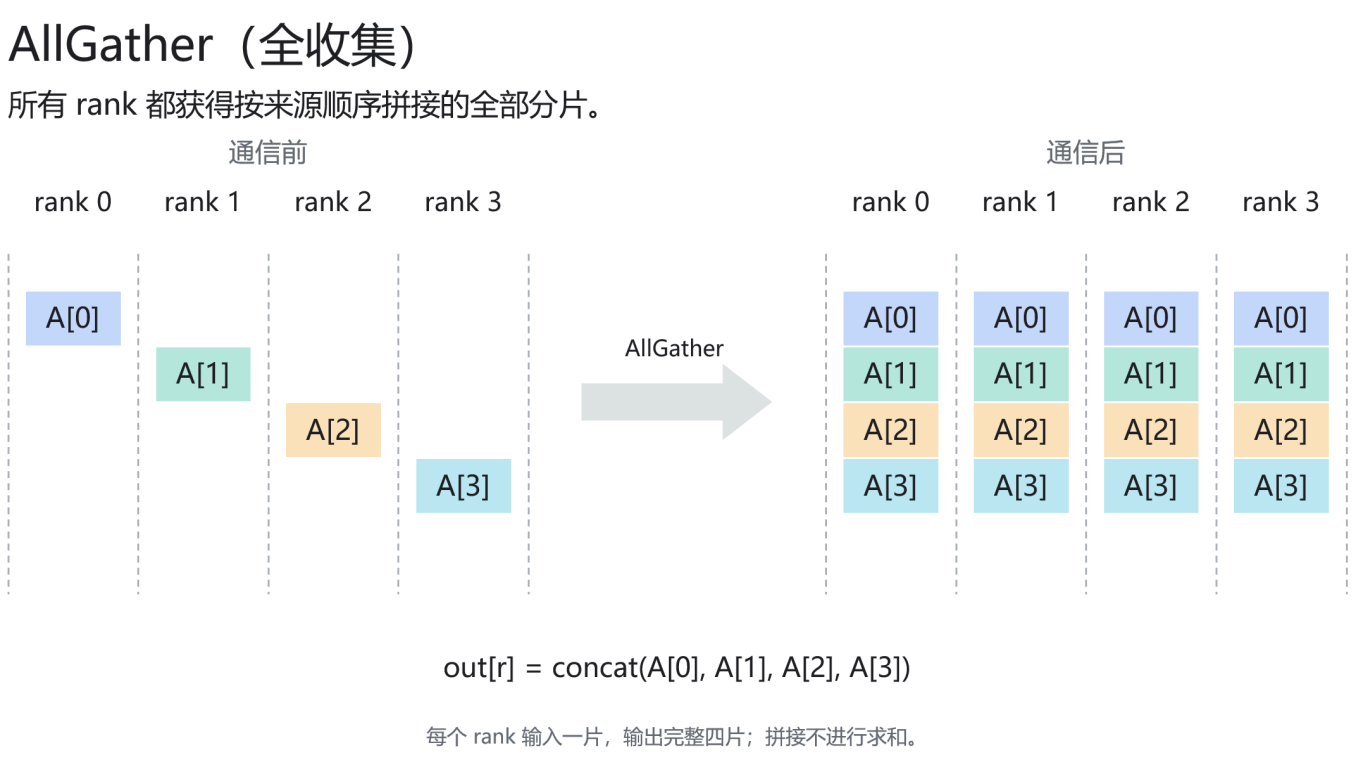
```
Gather后状态：
Rank0: [A0,A1,A2,A3, B0,B1,B2,B3, C0,C1,C2,C3, D0,D1,D2,D3] （root，完整拼接张量）
Rank1: [B0, B1, B2, B3] （数据不变）
Rank2: [C0, C1, C2, C3] （数据不变）
Rank3: [D0, D1, D2, D3] （数据不变）
```

代码块

```python
x = T([r, r + 10])
parts = None
if r == 0:
    parts = [torch.empty_like(x) for _ in range(p)]
dist.gather(x, gather_list=parts, dst=0)
if r == 0:
    print(r, torch.cat(parts).tolist())
```

大模型训练核心进阶通信原语

01 AllGather：全收集



核心定义：Gather的全对等版本。通信域内每个rank把自己持有的分片，发送给其他所有rank，最终**所有rank都拿到全部分片，拼接成完整的大张量**。

用途：FSDP/ZeRO-3 使用参数分片计算前临时收集参数；TP 或 SP 在后续算子需要完整激活时收集相应分片。通常不是每次都把整个大模型的所有参数一次性 AllGather，而是按模块、层或参数组进行通信。

数据流：

代码块

```
初始状态：
Rank0: [A0, A1, A2, A3]  (分片0
Rank1: [B0, B1, B2, B3]  (分片1)
Rank2: [C0, C1, C2, C3]  (分片2)
Rank3: [D0, D1, D2, D3]  (分片3)

AllGather后状态：
Rank0: [A0,A1,A2,A3, B0,B1,B2,B3, C0,C1,C2,C3, D0,D1,D2,D3]  (完整张量)
Rank1: [A0,A1,A2,A3, B0,B1,B2,B3, C0,C1,C2,C3, D0,D1,D2,D3]
Rank2: [A0,A1,A2,A3, B0,B1,B2,B3, C0,C1,C2,C3, D0,D1,D2,D3]
Rank3: [A0,A1,A2,A3, B0,B1,B2,B3, C0,C1,C2,C3, D0,D1,D2,D3]
```

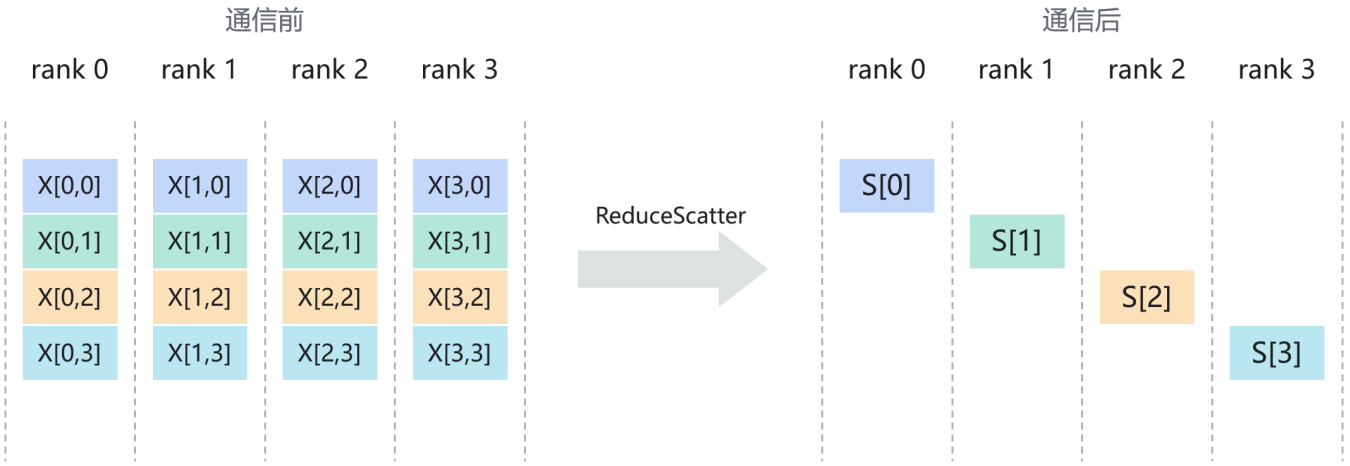
代码块

```
x = T([r, r + 10])
parts = [torch.empty_like(x) for _ in range(p)]
dist.all_gather(parts, x)
out = torch.cat(parts)
print(r, out.tolist())
```

02 ReduceScatter：规约分发

ReduceScatter（规约分发）

先对对应元素求和，再让每个 rank 保留结果的一片。



$$S[j] = X[0,j] + X[1,j] + X[2,j] + X[3,j]$$

rank j 仅得到 S[j]；同色表示相同分片位置；X[r,j] 是 rank r 的第 j 片。

核心定义：先做Reduce规约，再做Scatter分发。通信域内所有rank的输入数据，先按维度分片做规约，再把每个规约后的分片分发给对应的rank，最终**每个rank只拿到对应维度的规约后的分片**。

用途：分片梯度同步：汇总各数据副本的贡献后，每个 rank 只保留负责更新的梯度片；也用于 TP 结合 SP 时保持序列分片。

数据流：

代码块

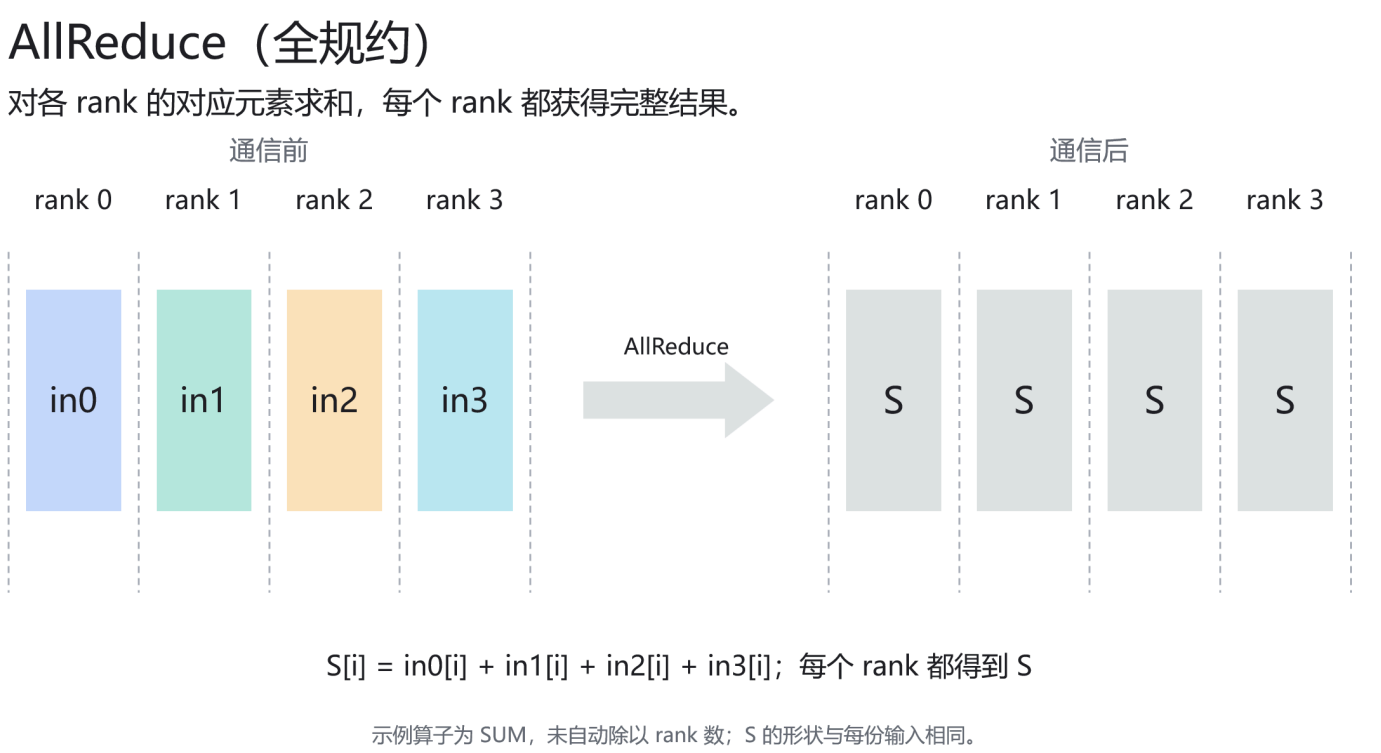
```
初始状态（每个rank的输入按4个维度分片，对应rank0~3）：
Rank0输入：[A0, A1, A2, A3] → 分片0:A0, 分片1:A1, 分片2:A2, 分片3:A3
Rank1输入：[B0, B1, B2, B3] → 分片0:B0, 分片1:B1, 分片2:B2, 分片3:B3
Rank2输入：[C0, C1, C2, C3] → 分片0:C0, 分片1:C1, 分片2:C2, 分片3:C3
Rank3输入：[D0, D1, D2, D3] → 分片0:D0, 分片1:D1, 分片2:D2, 分片3:D3

ReduceScatter操作：
1.  对分片0做SUM规约：A0+B0+C0+D0 → 分发给Rank0
2.  对分片1做SUM规约：A1+B1+C1+D1 → 分发给Rank1
3.  对分片2做SUM规约：A2+B2+C2+D2 → 分发给Rank2
4.  对分片3做SUM规约：A3+B3+C3+D3 → 分发给Rank3

最终状态：
Rank0：[A0+B0+C0+D0]  （仅分片0的规约结果）
Rank1：[A1+B1+C1+D1]  （仅分片1的规约结果）
Rank2：[A2+B2+C2+D2]  （仅分片2的规约结果）
Rank3：[A3+B3+C3+D3]  （仅分片3的规约结果）
```

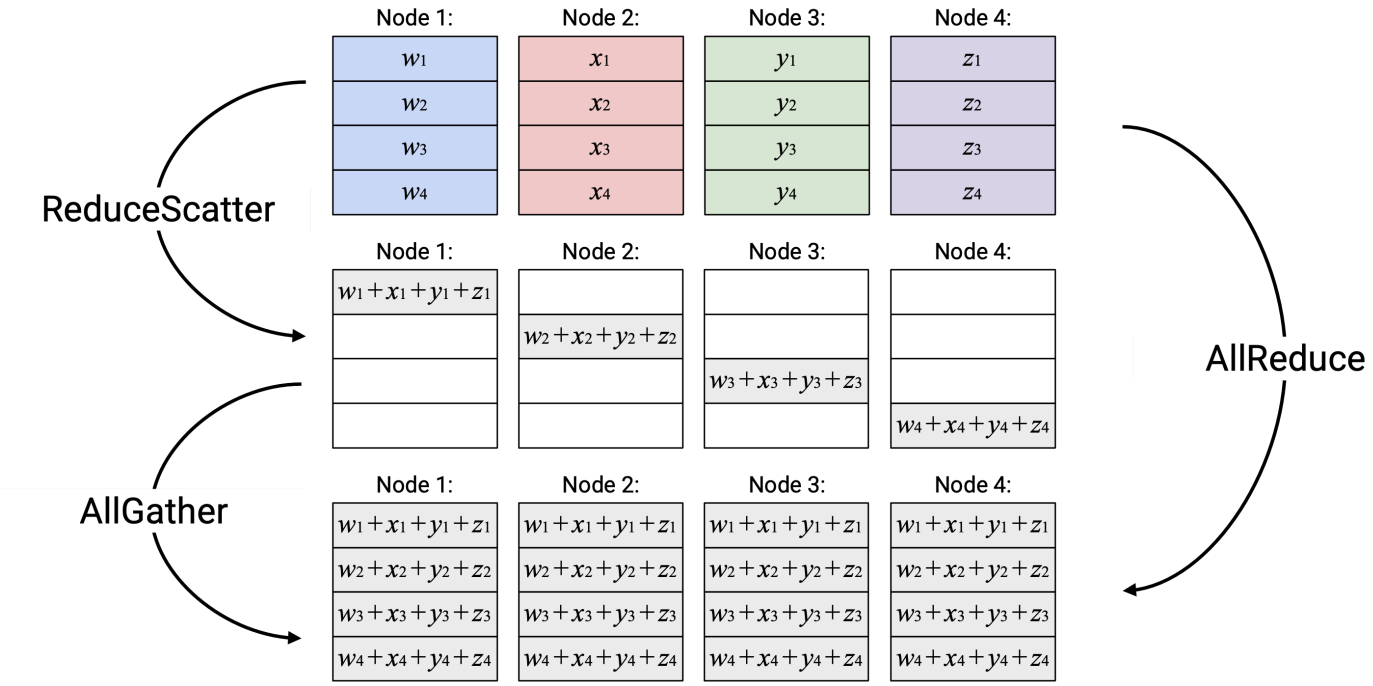
```
    x = T([r, r + 1, r + 2, r + 3])
    out = T([0])
    dist.reduce_scatter_tensor(
        out, x, op=dist.ReduceOp.SUM
    )
    print(r, out.tolist())
```

03 AllReduce：全规约



核心定义： Reduce的全对等版本。通信域内所有rank的输入数据，按指定规约操作聚合，最终所有rank都拿到完全相同的聚合结果。

$$\mathrm{AllReduce}(\mathrm{SUM}) \simeq \mathrm{ReduceScatter}(\mathrm{SUM}) + \mathrm{AllGather}\,.$$



用途：DDP 梯度同步；TP 行并行层把各 rank 的局部输出贡献相加，恢复同一逻辑输出

数据流：

代码块

```
初始状态:
Rank0: [1, 1, 1, 1]
Rank1: [2, 2, 2, 2]
Rank2: [3, 3, 3, 3]
Rank3: [4, 4, 4, 4]

AllReduce后状态（SUM）:
Rank0: [10, 10, 10, 10]  （1+2+3+4的聚合结果）
Rank1: [10, 10, 10, 10]
Rank2: [10, 10, 10, 10]
Rank3: [10, 10, 10, 10]
```

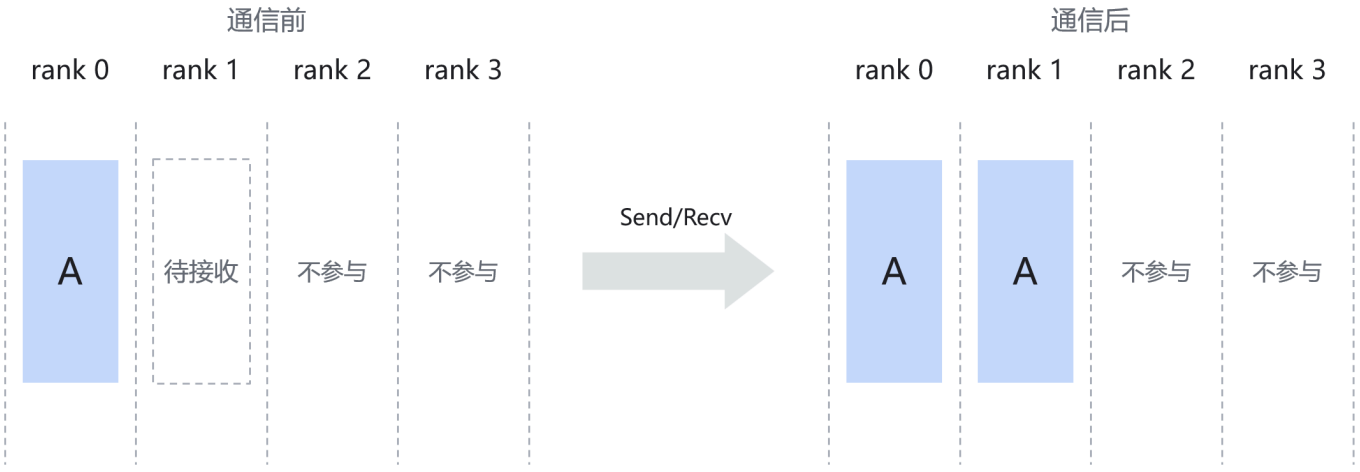
代码块

```
x = T([r, r + 1, r + 2, r + 3])
dist.all_reduce(x, op=dist.ReduceOp.SUM)
print(r, x.tolist())
```

04 Send/Recv：点对点通信

Send/Recv（点对点通信）

指定两个 rank 配对传输，其余 rank 不参与本次消息。



rank 0 → rank 1：recv = send = A

rank 0 保留发送数据；rank 2、3 不参与本次 P2P；此项不属于集合通信。

核心定义： Send / Recv 是两个 Rank 之间一对一的定向通信原语。发送方通过 Send 发送张量，接收方通过 Recv 将数据接收到预先分配的缓冲区中，常用于流水线并行（PP）中相邻 Stage 之间的激活与梯度传递。

两种核心模式：

1. **阻塞模式（Blocking）：** `send/recv` 调用会等待对应通信操作满足完成条件后才返回。如果对端没有提交匹配的通信操作，可能一直等待，导致计算资源闲置或死锁。
2. **异步模式（Non-blocking / Asynchronous）：** `isend/irecv` 提交通信请求后返回 `Work` 对象，无需在调用处等待整个通信完成，程序可以继续执行其他独立计算，并在需要时通过 `wait()` 建立同步依赖。Megatron-LM 的流水线并行支持这种通信方式，并可通过通信调度实现通信与计算重叠（Communication-Computation Overlap）。

注意： 异步通信不代表数据已经完成传输，也不代表一定能与计算重叠。实际效果取决于通信后端、CUDA Stream、计算依赖以及流水线调度策略。

用途：pp前向传播等

数据流：

代码块

```
初始状态:
Rank0: [A0, A1, A2, A3]
Rank1: []

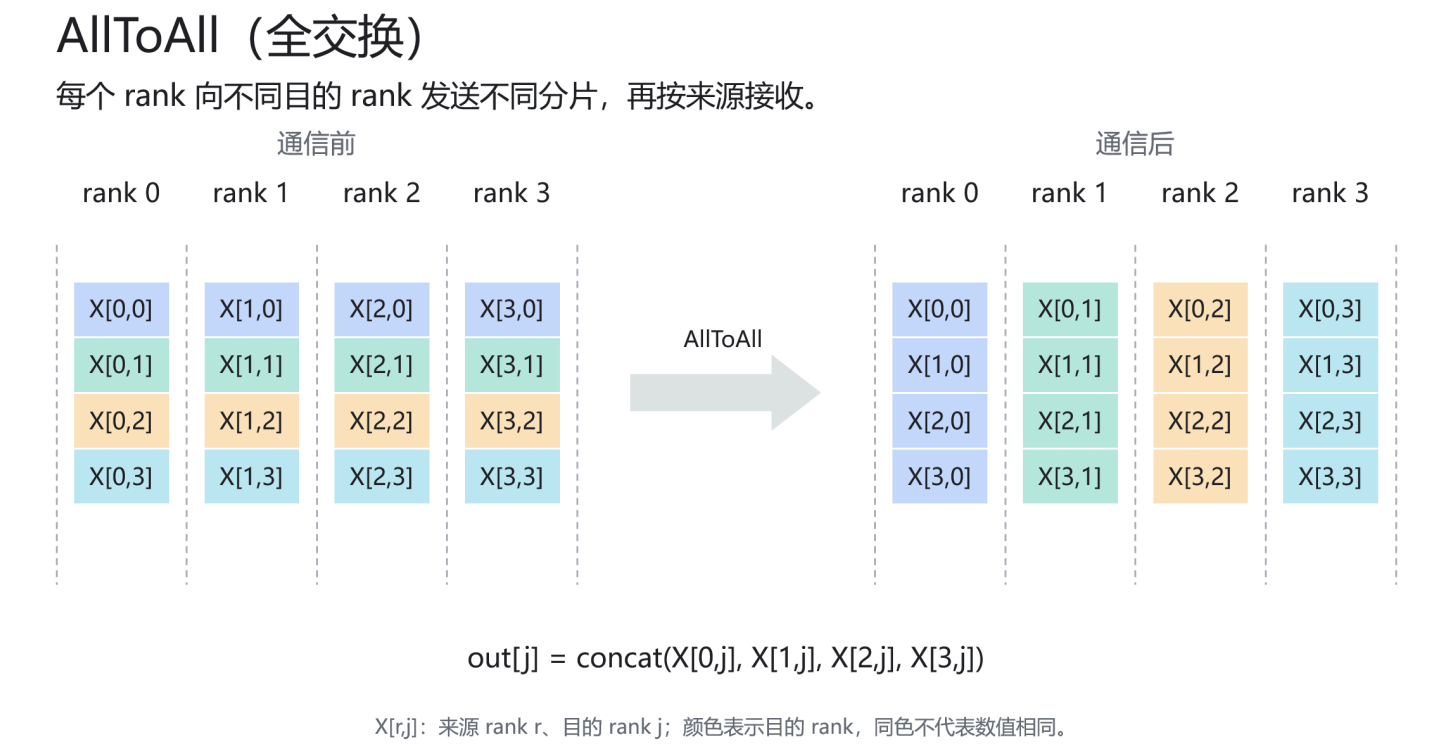
Send/Recv后状态:
Rank0: [A0, A1, A2, A3]
```

```
Rank1: [A0, A1, A2, A3]
```

代码块

```
if r == 0:
    x = T([100, 200])
    dist.send(x, dst=1)
elif r == 1:
    out = T([0, 0])
    dist.recv(out, src=0)
    print(r, out.tolist())
```

05 AllToAll：全交换



核心定义：通信域内每个rank，把自己的输入张量按rank数量切成分片，把第i个分片发送给第i个rank；同时接收所有其他rank发来的对应分片，最终拼接成输出张量。

用途：MoE 的 AllToAll dispatcher 把 token 送往对应专家，计算后交换回来

数据流：

代码块

```
初始状态：
Rank0: [A0, A1]  （分片0:A0留给自己，分片1:A1发给Rank1）
Rank1: [B0, B1]  （分片0:B0发给Rank0，分片1:B1留给自己）

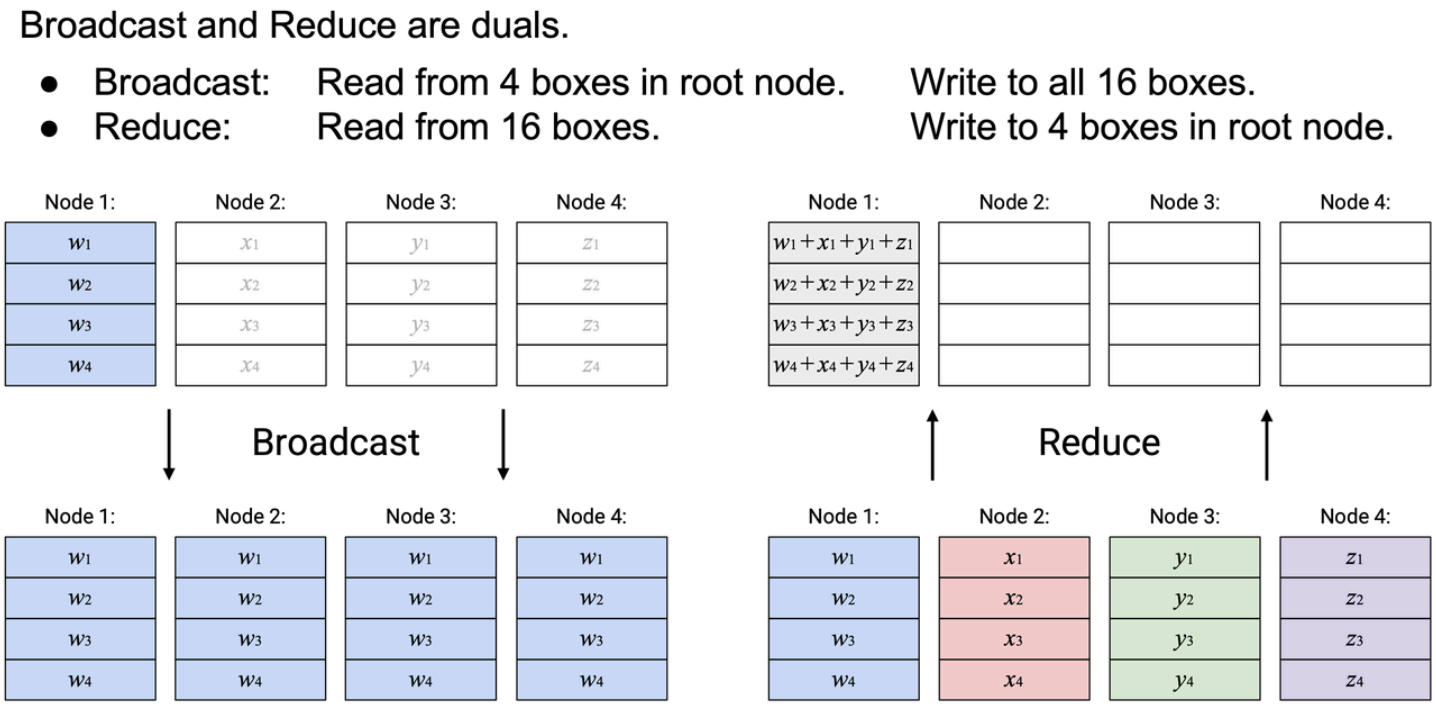
```

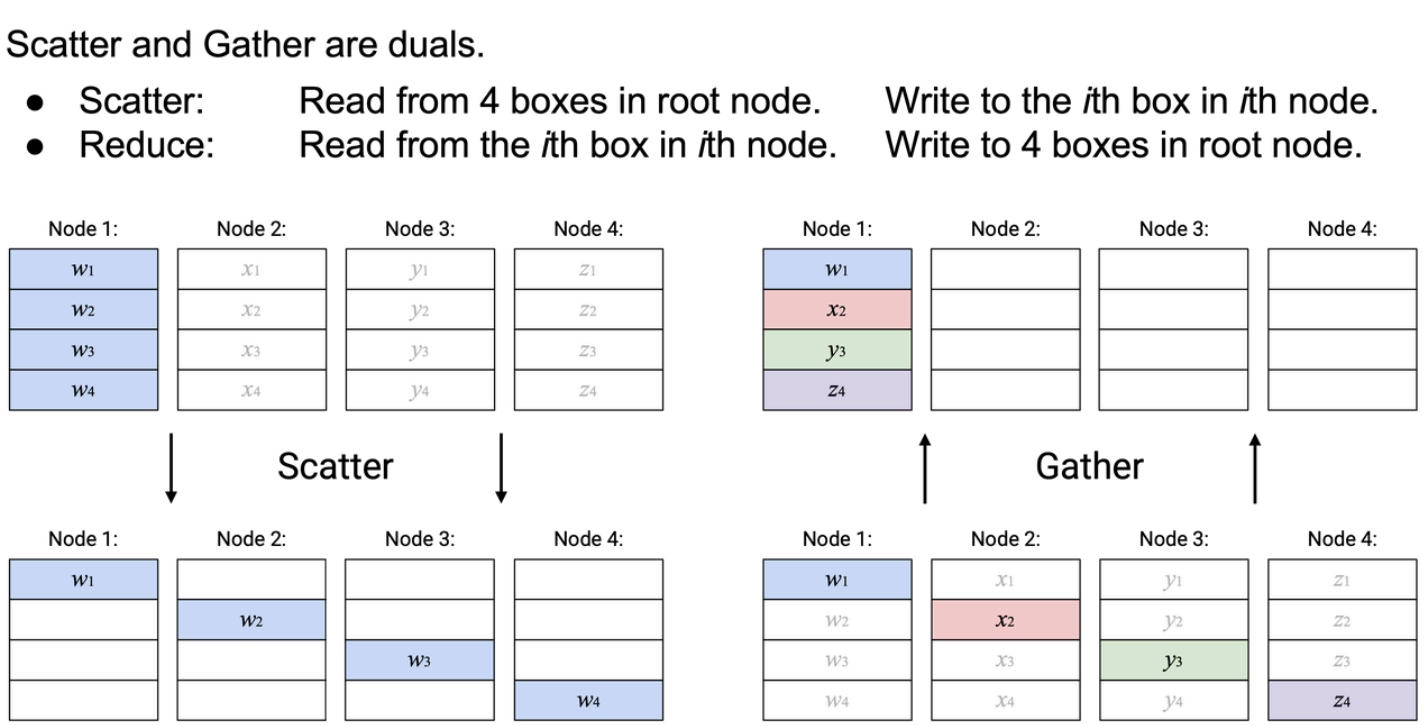
```
AllToAll后状态:
Rank0: [[A0],
        [B0]]
Rank1: [[A1],
        [B1]]
```

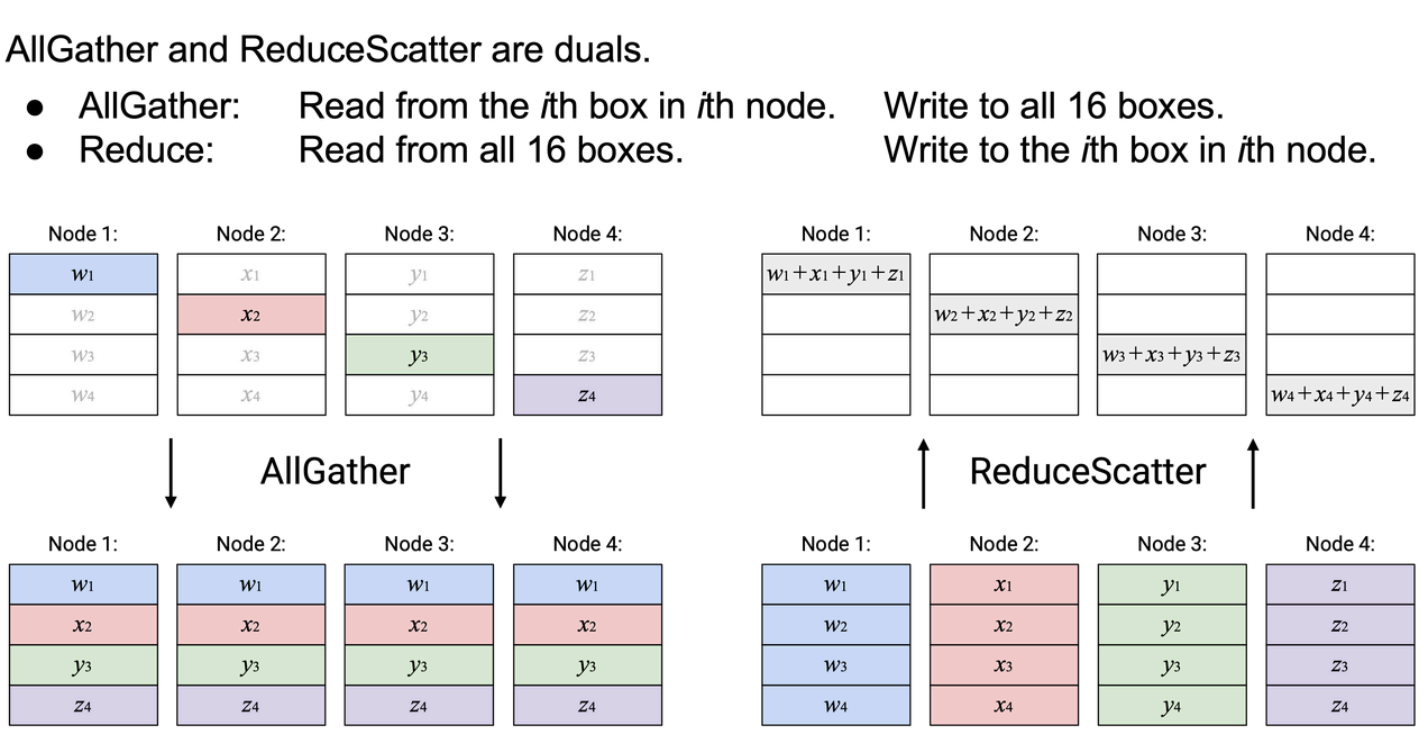
代码块

```
x = T([4 * r + j for j in range(p)])
out = torch.empty_like(x)
dist.all_to_all_single(out, x)
print(r, out.tolist())
```

原语之间的关系







第二部分

> 当一个大模型被拆分到多张 GPU 上训练时，前向传播、反向传播和参数更新分别会产生哪些通信？为什么必须通信？

表 1：主要并行策略总览

并行策略	如何拆分	典型通信原语	学习优先级
DP 数据并行	数据分片，模型复制	AllReduce	重点
TP 张量并行	层内权重矩阵分片	AllReduce、AllGather、ReduceScatter	核心
PP 流水线并行	按连续层划分 Stage	Send / Recv	重点
EP 专家并行	不同 Rank 存储不同专家	AllToAll / AllToAllV	进阶

表 2：前向与反向通信行为

并行策略	前向传播	反向传播	核心通信目的
DP	通常无逐层张量通信	AllReduce	同步梯度
TP：列并行	通常无通信，必要时 AllGather	输入梯度 AllReduce	合并输入梯度贡献
TP：行并行	AllReduce	通常无通信，必要时 AllGather	合并部分输出
PP	Send / Recv	Send / Recv	跨 Stage 传递激活和梯度
EP	AllToAll 分发 Token	AllToAll 等	将 Token 路由到专家

第一，DDP 不一定等到整个反向传播结束才调用 AllReduce。 PyTorch DDP 通常在梯度 Bucket 就绪后异步发起 AllReduce，从而让梯度通信与反向计算重叠。

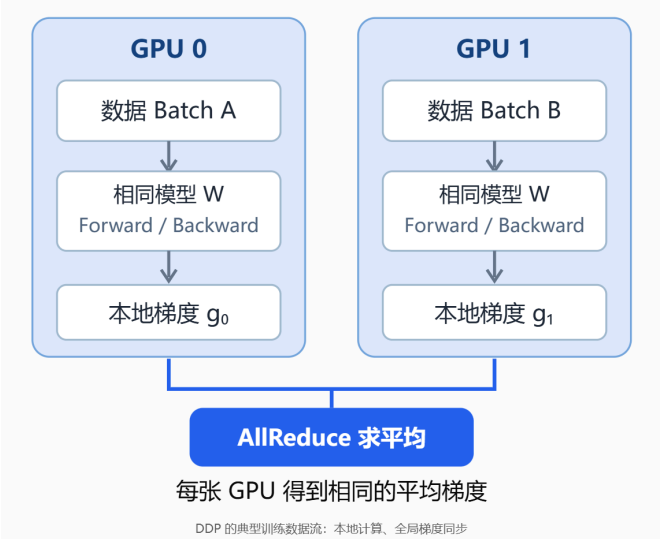
从抽象层次上讲：

$$\boxed{\text{并行策略} \rightarrow \text{通信需求} \rightarrow \text{通信原语} \rightarrow \text{通信算法}}$$

例如：

$$\text{DDP} \rightarrow \text{梯度同步} \rightarrow \text{AllReduce} \rightarrow \text{Ring AllReduce}$$

数据并行DP



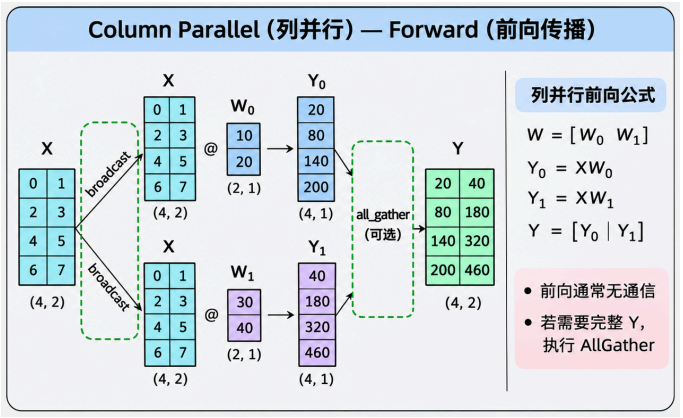
前向： 通常无 DP 组跨卡通信，各 Rank 处理自己的 Batch。

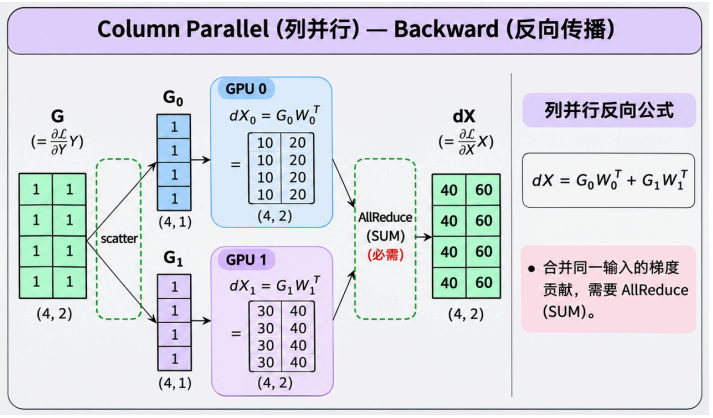
反向： 反向传播过程中使用 AllReduce 对梯度进行求和并平均

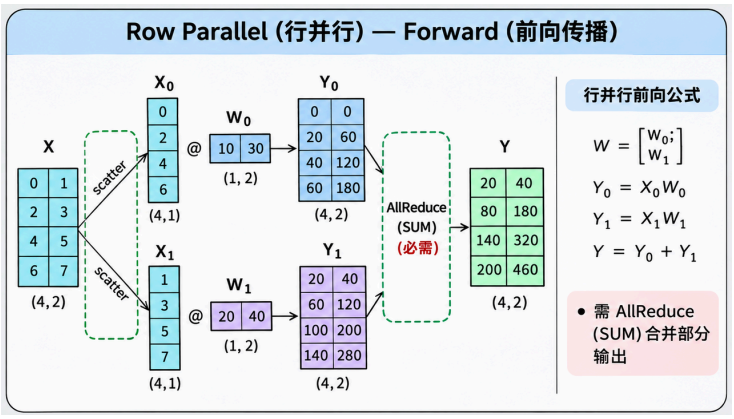
Q DDP中为什么初始化用broadcast，不是分片数据吗?

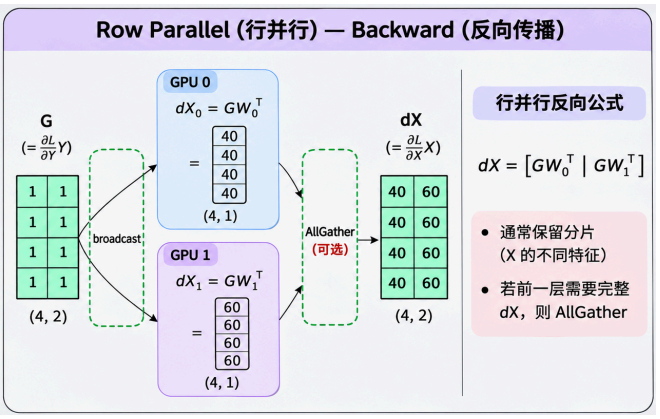
DDP 初始化时使用 Broadcast，同步的是模型参数和部分模型缓冲区，而不是训练数据。

张量并行TP









前向： 两个 Rank 分别计算 Y_0, Y_1 。如果下一层可以继续使用分片布局，则无需立刻通信；只有需要完整 Y 时才 AllGather。

反向： 本地可独立计算分片权重梯度；若各 Rank 都需要相同的完整输入梯度，则对各自算出的输入梯度贡献做 AllReduce 求和。

列并行：

- 相加：是对同一个x的梯度贡献
- X整个对两个路径都产生影响返回梯度故要相加

行并行：

- x分开两列，返回梯度要合并。
- gpu0算第一列的梯度，gpu1算第二列的梯度

$$
\begin{array}{|ll|}
\hline
\text{列并行：} & Y = [XW_0 \mid XW_1], \\
 & dX = G_0 W_0^T + G_1 W_1^T \\
\text{行并行：} & Y = X_0 W_0 + X_1 W_1, \\
 & dX = [GW_0^T \mid GW_1^T] \\
\hline
\end{array}
$$

只要一个输入元素通过多条路径影响损失，反向传播时就必须把所有路径传回来的梯度贡献相加。

看 GPU 算出来的梯度，对应的是不是同一个输入元素。

- 是同一个输入元素的不同贡献：必须相加（列并行反向，AllReduce）。
- 是不同输入元素各自的梯度：不能相加，只需保留分片或拼接（行并行反向）。

Q：前向传播究竟需不需要通信？

答案：如果每个 Rank 已经拥有完整输入 X，并且允许输出保持分片状态，那么前向传播不需要通信。

因为 GPU 0 和 GPU 1 计算的是不同的输出特征，没有需要跨 GPU 求和的部分。

但如果后续计算要求每张 GPU 都拥有完整的 Y ，则需要：

$Y = \mathrm{AllGather}(Y_0, Y_1)$

这对应 Megatron `ColumnParallelLinear` 的 `gather_output` 配置。默认可保持输出分片，而不立即执行 AllGather。

流水线并行PP

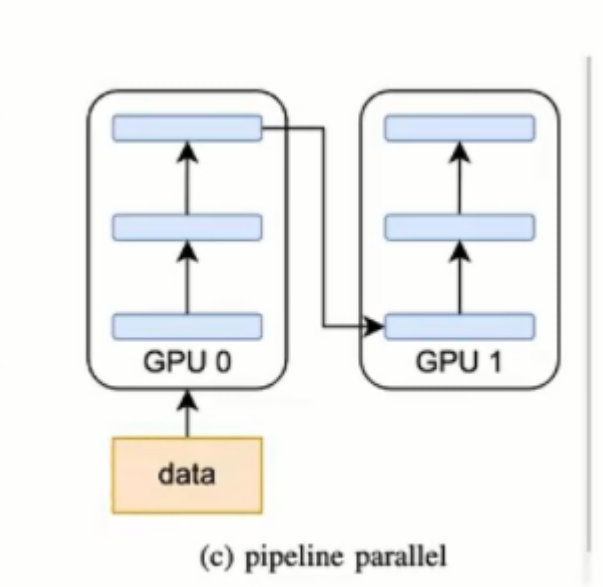
如何切分：按连续层把模型分成多个 Stage。例如 GPU0 保存前半部分层 f_0 ，GPU1 保存后半部分层 f_1 ：

$$a = f_0(X), \qquad Y = f_1(a).$$

GPU 0 只需要将自己的输出交给下一阶段 GPU 1，不需要让整个通信组的所有 GPU 都拥有这份数据。

前向传递激活值

反向传播传递激活梯度G



(c) pipeline parallel

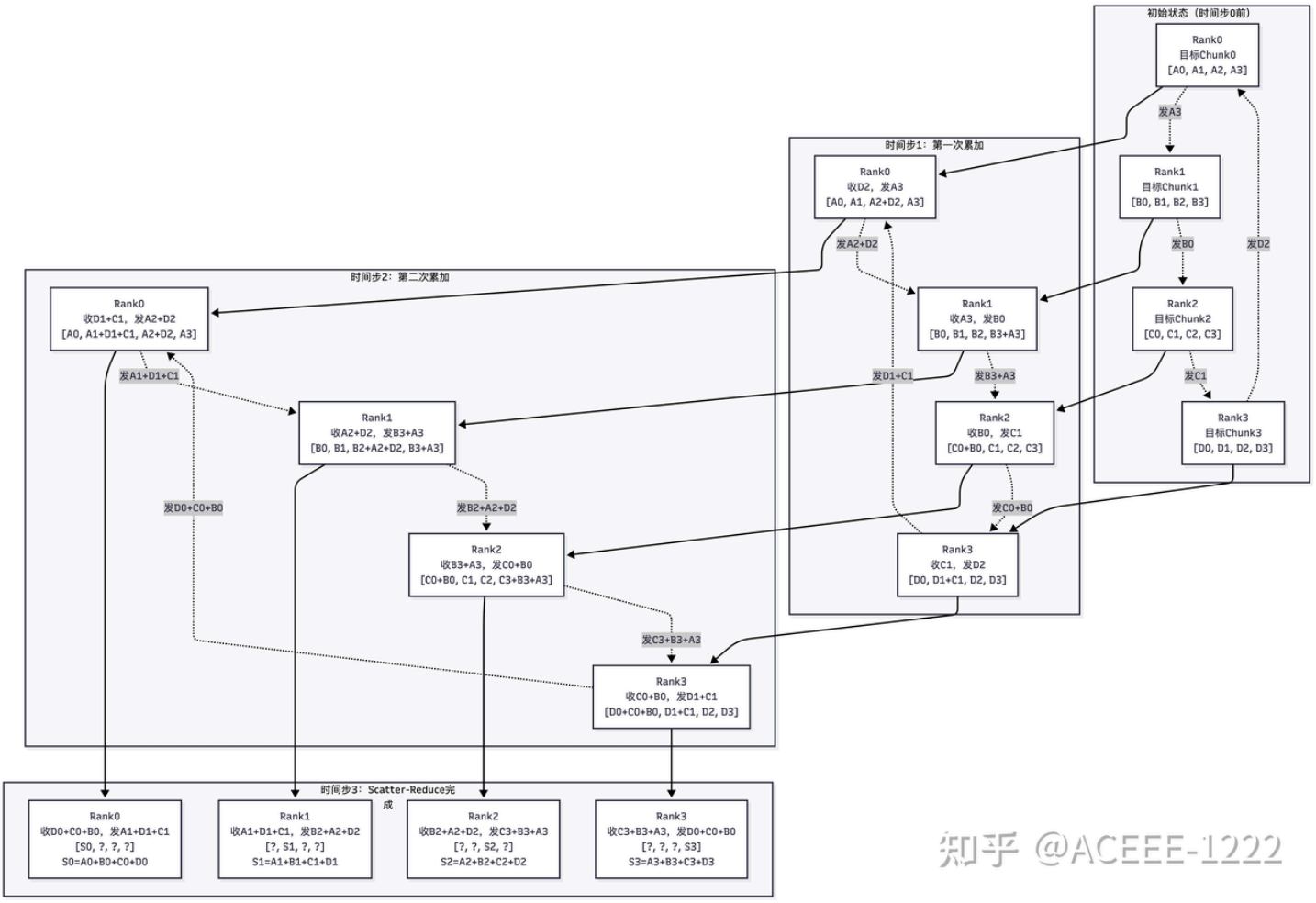
Q ReduceScatter 是不是就是先 Reduce、再 Scatter

A: ReduceScatter 在数学结果上等价于先 Reduce、再 Scatter，但实际通信时，可以一边传输数据、一边完成归约和分片，而不需要先生成一份完整的归约结果。

Rank 0 承担了大量数据接收和发送任务。

其他 GPU 的通信能力没有得到充分利用。

所有数据集中到 Rank 0，容易产生带宽瓶颈。



知乎 @ACEEE-1222

代码块

```
================================================================================
【第一阶段：Scatter-Reduce 逐时间步演示】
================================================================================
初始状态（时间步0前）：
Rank0: [A0, A1, A2, A3] 目标Chunk0
Rank1: [B0, B1, B2, B3] 目标Chunk1
Rank2: [C0, C1, C2, C3] 目标Chunk2
Rank3: [D0, D1, D2, D3] 目标Chunk3
环形拓扑：Rank0 ↔ Rank1 ↔ Rank2 ↔ Rank3 ↔ Rank0

--------------------------------------------------------------------------------
时间步1（每个节点向右发送目标Chunk的前一个Chunk）：
- Rank0收D2，发A3；Chunk2 = A2 + D2
- Rank1收A3，发B0；Chunk3 = B3 + A3
- Rank2收B0，发C1；Chunk0 = C0 + B0
- Rank3收C1，发D2；Chunk1 = D1 + C1

状态：
Rank0: [A0, A1, A2+D2, A3]
Rank1: [B0, B1, B2, B3+A3]
Rank2: [C0+B0, C1, C2, C3]
```

```
Rank3: [D0, D1+C1, D2, D3]

──────────────────────────────────────────────────────────────────────────────
──────
时间步2（每个节点发送上一步刚刚更新的Chunk）：
- Rank0收D1+C1，发A2+D2；Chunk1 = A1 + D1 + C1
- Rank1收A2+D2，发B3+A3；Chunk2 = B2 + A2 + D2
- Rank2收B3+A3，发C0+B0；Chunk3 = C3 + B3 + A3
- Rank3收C0+B0，发D1+C1；Chunk0 = D0 + C0 + B0

状态：
Rank0: [A0, A1+D1+C1, A2+D2, A3]
Rank1: [B0, B1, B2+A2+D2, B3+A3]
Rank2: [C0+B0, C1, C2, C3+B3+A3]
Rank3: [D0+C0+B0, D1+C1, D2, D3]

──────────────────────────────────────────────────────────────────────────────
──────
时间步3（Scatter-Reduce完成，发送上一步更新的Chunk，补齐最后一块拼图）：
- Rank0收D0+C0+B0，发A1+D1+C1；Chunk0 = A0 + D0 + C0 + B0（完整规约）
- Rank1收A1+D1+C1，发B2+A2+D2；Chunk1 = B1 + A1 + D1 + C1（完整规约）
- Rank2收B2+A2+D2，发C3+B3+A3；Chunk2 = C2 + B2 + A2 + D2（完整规约）
- Rank3收C3+B3+A3，发D0+C0+B0；Chunk3 = D3 + C3 + B3 + A3（完整规约）

【Scatter-Reduce最终状态】：
Rank0: [S0, ?, ?, ?]  （S0 = A0+B0+C0+D0，仅持有S0）
Rank1: [?, S1, ?, ?]  （S1 = A1+B1+C1+D1，仅持有S1）
Rank2: [?, ?, S2, ?]  （S2 = A2+B2+C2+D2，仅持有S2）
Rank3: [?, ?, ?, S3]  （S3 = A3+B3+C3+D3，仅持有S3）
```

参考资料

https://zhuanlan.zhihu.com/p/2013649391051896797

https://zhuanlan.zhihu.com/p/1937826285264011929

https://zhuanlan.zhihu.com/p/1940505073496068488

https://textbook.cs168.io/beyond-client-server/collective-operations.html